

You are an AI engineering agent. Your job is to build the ElevatorIQ backend exactly as specified in this document. Work through each phase sequentially. Do not skip phases. Complete every checkpoint before proceeding. When in doubt, ask before building — architecture decisions are cheaper to change before the code is written.

Project Context

ElevatorIQ is a structured elevator commercial intelligence platform. Users upload elevator maintenance invoices, maintenance bids, and modernization proposals. The system analyzes them using the Claude AI API and returns a structured PDF report. Every analysis also extracts anonymized structured data into a database that builds benchmark intelligence over time — making future analyses smarter.

Component	Decision
Language	Node.js (preferred) or Python FastAPI
Database	PostgreSQL — mandatory, no substitutions
AI API	Anthropic Claude API (claude-sonnet-4-6)
Object Storage	AWS S3 or Cloudflare R2
Worker Queue	Bull (Node) or Celery (Python)
Email	Postmark, SendGrid, or Resend
PDF Generation	Puppeteer (Node) or WeasyPrint (Python)
Dev Environment	Local Postgres + .env for all secrets

■ **CRITICAL** Never hardcode API keys. All secrets go in .env. Never commit .env to GitHub.

1.1 Create the database

Create a local PostgreSQL database named `elevatoriq_dev`. Run the following setup:

TERMINAL

```
createdb elevatoriq_dev
psql elevatoriq_dev
```

1.2 Run schema_v1.sql

The file `schema_v1.sql` (provided separately) contains all 10 table definitions. Run it against the database:

TERMINAL

```
psql elevatoriq_dev < schema_v1.sql
```

Confirm all 10 tables were created:

VERIFY IN PSQL

```
\dt
-- Expected: customers, cases, documents, reports, extractions_raw,
-- facts_labor, facts_line_items, facts_parts, facts_contract_terms, benchmarks
```

1.3 Verify immutability rules on extractions_raw

The schema applies NO UPDATE / NO DELETE rules to `extractions_raw`. Test this:

VERIFY IN PSQL

```
INSERT INTO extractions_raw (case_id, module, raw_json)
VALUES (gen_random_uuid(), 'A', '{"test": true}');

-- This should silently do nothing (rule blocks it):
UPDATE extractions_raw SET module = 'B' WHERE module = 'A';

-- Confirm row is unchanged:
SELECT module FROM extractions_raw LIMIT 1; -- should still be 'A'
```

✓ CHECKPOINT

All 10 tables exist. `extractions_raw` rejects updates. Proceed to Phase 2.

2.1 Initialize project

TERMINAL — NODE

```
mkdir elevatoriq-backend && cd elevatoriq-backend
npm init -y
npm install express pg multer aws-sdk @anthropic-ai/sdk
npm install bull ioredis nodemailer dotenv uuid ajv
npm install puppeteer
npm install --save-dev nodemon
```

2.2 Create .env file

■ CRITICAL

Never commit this file. Confirm .env is in .gitignore before proceeding.

.env

```
DATABASE_URL=postgresql://localhost:5432/elevatoriq_dev
ANTHROPIC_API_KEY=your_key_here
AWS_BUCKET=your_bucket_name
AWS_REGION=us-east-1
AWS_ACCESS_KEY_ID=your_key
AWS_SECRET_ACCESS_KEY=your_secret
EMAIL_PROVIDER_API_KEY=your_key
FROM_EMAIL=reports@elevatoriq.ai
PORT=3001
```

2.3 Project file structure to create

FILE STRUCTURE

```
elevatoriq-backend/
src/
db.js -- Postgres connection pool
routes/
cases.js -- POST /api/cases, POST /api/cases/:id/run
documents.js -- POST /api/cases/:id/documents
reports.js -- GET /api/reports/download/:token
workers/
analysisWorker.js -- Main job processor
services/
claudeService.js -- Claude API call + dual-output parser
benchmarkService.js -- Benchmark retrieval query
pdfService.js -- PDF generation
```

```
emailService.js -- Email delivery
storageService.js -- S3 upload/download
validation/
extractionSchema.js -- AJV schema validator
schema_v1.sql
.env
index.js
```

✓ **CHECKPOINT**

Project initialized. .env exists and is gitignored. File structure created. Proceed to Phase 3.

3.1 POST /api/cases — Create a case

This endpoint creates a new case record and returns the case_id. The frontend sends review_type, state, and market.

src/routes/cases.js

```
// routes/cases.js
router.post('/', async (req, res) => {
  const { customer_id, review_type, state, market } = req.body;
  const module = getModule(review_type); // 'A', 'B', or 'C'
  const result = await db.query(
    `INSERT INTO cases (customer_id, review_type, module, state, market)
    VALUES ($1,$2,$3,$4,$5) RETURNING id`,
    [customer_id || null, review_type, module, state, market]
  );
  res.json({ case_id: result.rows[0].id, status: 'pending' });
});

function getModule(review_type) {
  if (review_type === 'invoice_review') return 'A';
  if (review_type === 'contract_coverage') return 'A';
  if (review_type === 'maintenance_bid_comparison') return 'C';
  return 'B'; // modernization_comparison, single_modernization
}
```

3.2 POST /api/cases/:id/documents — Upload files

Use multer for multipart upload. Store file in S3. Insert document record in DB.

src/routes/documents.js

```
// routes/documents.js
router.post('/:id/documents', upload.single('file'), async (req, res) => {
  const { id } = req.params;
  const { file_type } = req.body; // optional tag from user
  const storagePath = await storageService.upload(req.file, id);
  const autoDetected = !file_type;
  const detectedType = file_type || detectType(req.file.originalname);
  const result = await db.query(
    `INSERT INTO documents (case_id, file_name, file_type, storage_path, auto_detected)
    VALUES ($1,$2,$3,$4,$5) RETURNING id`,
    [id, req.file.originalname, detectedType, storagePath, autoDetected]
  );
  res.json({ document_id: result.rows[0].id, auto_detected_type: detectedType });
});
```

```
});
```

✓ **CHECKPOINT**

Can POST a case, upload a file, and see both records in the database. S3 bucket contains the uploaded file.

This is the most critical phase. The dual-output requirement — REPORT_BODY + EXTRACTION_JSON — is the foundation of the entire flywheel. Get this right before building anything else downstream.

4.1 Benchmark retrieval (inject before Claude call)

src/services/benchmarkService.js

```
// services/benchmarkService.js
async function getBenchmarkContext(state, equipment_type) {
  const result = await db.query(`
    SELECT DISTINCT ON (dimension) dimension, median_value, p25_value,
    p75_value, sample_size, benchmark_version
    FROM benchmarks
    WHERE published = TRUE
    AND state = $1 AND equipment_type = $2
    ORDER BY dimension, benchmark_version DESC
    `, [state, equipment_type]);

  if (!result.rows.length) return '';

  let ctx = '[BENCHMARK CONTEXT]\n';
  result.rows.forEach(r => {
    ctx += `${r.dimension}: ${r.median_value} median `;
    ctx += `(p25=${r.p25_value}, p75=${r.p75_value}, `;
    ctx += `n=${r.sample_size}, v=${r.benchmark_version})\n`;
  });
  return ctx;
}
```

4.2 Claude API call with dual-output prompt

src/services/claudeService.js

```
// services/claudeService.js
const Anthropic = require('@anthropic-ai/sdk');
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

async function analyze(documentText, reviewType, benchmarkContext) {
  const systemPrompt = getRulebook(); // load rulebook_v1.1 text

  const userPrompt = `
    ${benchmarkContext}

    Review Type: ${reviewType}

    DOCUMENTS:
  `;
}
```

```
${documentText}
```

After your analysis, output TWO sections exactly as follows:

```
---REPORT_BODY---
```

[Your full structured report here]

```
---EXTRACTION_JSON---
```

[Valid JSON only, conforming to schema v1. No prose. No markdown.]

```
`;
```

```
const response = await client.messages.create({
  model: 'claude-sonnet-4-6',
  max_tokens: 4096,
  system: systemPrompt,
  messages: [{ role: 'user', content: userPrompt }]
});

const raw = response.content[0].text;
const [reportBody, jsonPart] = raw.split('---EXTRACTION_JSON---');
return {
  reportBody: reportBody.replace('---REPORT_BODY---', '').trim(),
  extractionJson: jsonPart ? jsonPart.trim() : null
};
}
```

4.3 Load the Rulebook

Save the full contents of rulebook_v1.1.md as a .txt file in the project. Load it at runtime:

```
src/services/claudeService.js
```

```
// Inside claudeService.js
const fs = require('fs');
const path = require('path');

function getRulebook() {
  return fs.readFileSync(
    path.join(__dirname, '../..../rulebook_v1.1.txt'), 'utf8'
  );
}
```

✓ CHECKPOINT

claudeService.analyze() returns both reportBody and extractionJson. Test with a sample document before proceeding.

5.1 AJV Schema Validator

Validate extractionJson before any DB insert. If it fails, log it but do NOT block the customer from getting their report.

src/validation/extractionSchema.js

```
// validation/extractionSchema.js
const Ajv = require('ajv');
const ajv = new Ajv();

const schema = {
  type: 'object',
  required: ['module', 'state', 'market', 'equipment_type', 'confidence_overall'],
  properties: {
    schema_version: { type: 'string' },
    case_id: { type: 'string' },
    module: { type: 'string', enum: ['A', 'B', 'C'] },
    state: { type: 'string', maxLength: 2 },
    market: { type: 'string' },
    equipment_type: { type: 'string' },
    contract_type: { type: 'string' },
    unit_count: { type: 'integer' },
    confidence_overall: { type: 'string', enum: ['high', 'medium', 'low'] },
    // ... add all fields from schema v1
  }
};

module.exports = { validate: ajv.compile(schema) };
```

5.2 Insert into extractions_raw

■ **CRITICAL**

Strip all vendor names, building names, and owner names BEFORE inserting. Store only state + market for geography.

src/workers/analysisWorker.js

```
// Inside analysisWorker.js
const { validate } = require('../validation/extractionSchema');

async function saveExtraction(caseId, parsed) {
  let json;
  try {
    json = JSON.parse(parsed.extractionJson);
  } catch (e) {
    console.error('JSON parse failed for case', caseId, e);
  }
}
```

```
return; // do not block
}

const valid = validate(json);
if (!valid) {
  console.error('Schema validation failed', validate.errors);
  return; // do not block
}

// Insert into extractions_raw (append-only)
await db.query(
  `INSERT INTO extractions_raw
(case_id, module, state, market, equipment_type, contract_type,
unit_count, confidence_overall, benchmark_version, raw_json)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)`,
[caseId, json.module, json.state, json.market, json.equipment_type,
json.contract_type, json.unit_count, json.confidence_overall,
json.benchmark_version, json]
);
}
```

✓ CHECKPOINT

extractionJson is validated and inserted into extractions_raw. A bad JSON does not crash the job.

6.1 Generate PDF from reportBody

src/services/pdfService.js

```
// services/pdfService.js
const puppeteer = require('puppeteer');

async function generatePDF(reportBody, caseId) {
  const html = wrapInHTML(reportBody); // convert markdown-style to HTML
  const browser = await puppeteer.launch({ args: ['--no-sandbox'] });
  const page = await browser.newPage();
  await page.setContent(html, { waitUntil: 'networkidle0' });
  const pdf = await page.pdf({
    format: 'Letter',
    margin: { top: '0.75in', bottom: '0.75in', left: '0.75in', right: '0.75in' },
    printBackground: true
  });
  await browser.close();

  // Upload PDF to S3
  const key = `reports/${caseId}.pdf`;
  await storageService.uploadBuffer(pdf, key, 'application/pdf');
  return key;
}
```

6.2 Email delivery

■ CRITICAL

Never attach or send extractionJson in the email. PDF only.

src/services/emailService.js

```
// services/emailService.js
async function sendReport(toEmail, pdfBuffer, reviewType, downloadToken) {
  const downloadUrl =
    `https://elevatoriq.ai/api/reports/download/${downloadToken}`;

  await emailProvider.send({
    to: toEmail,
    from: process.env.FROM_EMAIL,
    subject: 'Your ElevatorIQ Analysis is Ready',
    html: `
Your ${reviewType} report is ready.
Your structured analysis is attached as a PDF.
Download Report (link expires in 7 days)
`
  });
}
```

This analysis evaluates contractual and commercial structure only.

```
`  
attachments: [{ filename: 'ElevatorIQ_Report.pdf', content: pdfBuffer }]  
});  
}
```

✓ **CHECKPOINT**

PDF is generated, uploaded to S3, and emailed with attachment. Download link works. No JSON in email.

This is the master job that ties all services together. It runs when POST `/api/cases/:id/run` is called.

`src/workers/analysisWorker.js`

```
// workers/analysisWorker.js
async function processCase(caseId) {
  try {
    // 1. Load case + documents from DB
    const caseRow = await db.query('SELECT * FROM cases WHERE id=$1',[caseId]);
    const docs = await db.query('SELECT * FROM documents WHERE case_id=$1',[caseId]);

    // 2. Extract text from each document (download from S3, parse PDF)
    const texts = await Promise.all(docs.rows.map(d => extractText(d.storage_path)));
    const combinedText = texts.join('\n\n--- NEXT DOCUMENT ---\n\n');

    // 3. Load benchmark context
    const benchmarks = await benchmarkService.getBenchmarkContext(
      caseRow.rows[0].state, caseRow.rows[0].equipment_type
    );

    // 4. Call Claude API
    const result = await claudeService.analyze(
      combinedText, caseRow.rows[0].review_type, benchmarks
    );

    // 5. Validate + save extraction (non-blocking on failure)
    await saveExtraction(caseId, result);

    // 6. Generate PDF
    const pdfKey = await pdfService.generatePDF(result.reportBody, caseId);

    // 7. Generate download token + save report record
    const token = require('uuid').v4();
    await db.query(
      `INSERT INTO reports (case_id, storage_path, download_token) VALUES ($1,$2,$3)`,
      [caseId, pdfKey, token]
    );

    // 8. Send email
    const pdfBuffer = await storageService.download(pdfKey);
    const customerEmail = await getCustomerEmail(caseId);
    await emailService.sendReport(
      customerEmail, pdfBuffer, caseRow.rows[0].review_type, token
    );

    // 9. Mark case complete
    await db.query(
```

```
`UPDATE cases SET status='complete', completed_at=NOW() WHERE id=$1`,  
[caseId]  
);  
  
} catch (err) {  
  console.error('Worker failed for case', caseId, err);  
  await db.query(`UPDATE cases SET status='failed' WHERE id=$1`, [caseId]);  
}  
}
```

✓ **CHECKPOINT**

Full end-to-end test: upload a real PDF, trigger analysis, receive email with PDF report. Check DB for `extractions_raw` row.

Run this job nightly once you have data. At early stage, run manually.

src/workers/aggregationJob.js

```
// workers/aggregationJob.js
async function runAggregation() {
  const VERSION = await getNextVersion(); // e.g. '1.1'
  const MIN_SAMPLE = 25;

  // Labor rates
  const labor = await db.query(`
    SELECT state, market, equipment_type,
    COUNT(*) as n,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY rate_regular) as median,
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY rate_regular) as p25,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY rate_regular) as p75
  FROM facts_labor
  WHERE rate_regular IS NOT NULL
  GROUP BY state, market, equipment_type
  `);

  for (const row of labor.rows) {
    await db.query(`
      INSERT INTO benchmarks
      (benchmark_version, dimension, state, market, equipment_type,
      sample_size, median_value, p25_value, p75_value, published)
      VALUES ($1, 'labor_regular', $2, $3, $4, $5, $6, $7, $8, $9)`,
      [VERSION, row.state, row.market, row.equipment_type,
      row.n, row.median, row.p25, row.p75, row.n >= MIN_SAMPLE]
    );
  }

  // Repeat for: overtime_multiplier, base_price_per_unit,
  // escalation_percent, material_markup_percent, warranty_months
  console.log('Aggregation complete. Version:', VERSION);
}
```

✓ CHECKPOINT

Aggregation job runs without error. benchmarks table has rows with published=TRUE where sample_size >= 25.

The frontend (ElevatorIQ_Landing_v6.jsx) is already deployed. It needs to call your backend.

9.1 CORS setup

src/index.js

```
// index.js
const cors = require('cors');
app.use(cors({ origin: 'https://elevatoriq.ai' }));
```

9.2 Frontend API calls to wire up

Frontend Action	Backend Endpoint
User clicks Run Analysis	POST /api/cases
Each file upload	POST /api/cases/:id/documents
Submit button final click	POST /api/cases/:id/run
Polling for status	GET /api/cases/:id/status
Download link	GET /api/reports/download/:token

9.3 Deploy backend

DEPLOYMENT

```
# Recommended: Railway.app or Render.com for fast deployment
# 1. Push backend to its own GitHub repo
# 2. Connect to Railway/Render
# 3. Set all .env variables in the platform dashboard
# 4. Point frontend API calls to the deployed backend URL
```

✓ CHECKPOINT

Full production test: submit from elevatoriq.ai, receive email with PDF, see DB records created.

Final Production Checklist

Item	Verified?
All 10 DB tables exist in production Postgres	■
extractions_raw rejects UPDATE and DELETE	■
Claude API returns dual output in every test	■
Bad JSON does not crash the worker job	■
PDF attaches correctly to email	■
extractionJson never sent to customer	■
Download token expires after 7 days	■
All secrets in env vars, not in code	■
Aggregation job scheduled (cron)	■
Benchmark retrieval injects into Claude prompt	■
Frontend wired to production backend URL	■

When all checkboxes are checked, ElevatorIQ is live as a full intelligence platform. Every upload feeds the flywheel. Every future analysis gets smarter.